
Poplog and Pop-11

A Working Introduction

The two-level virtual machine, the core language,
the four hosted front-ends, and the C interface

This book accompanies the Poplog source tree at github.com/IoTone/poplog — a maintained fork of the Poplog system originally developed at the University of Sussex. Every Pop-11, Forth, Prolog, Common Lisp and Standard ML listing in it was executed against that tree on Apple Silicon (macOS, arm64) while the text was written; the outputs shown are the outputs obtained.

Full in-tree documentation — 900+ `HELP`, `TEACH` and `REF` files — is browsable at <https://iotone.github.io/poplog/>.

© The Poplog contributors. Poplog is free software; this document is distributed with the source tree under the same terms.

Contents

1	What Poplog is, and why it is unusual	1	5	The hosted languages	13
1.1	The two-level virtual machine	1	5.1	Prolog	13
1.2	Incremental compilation, measured	1	5.2	Common Lisp	14
1.3	The saved image	2	5.3	Standard ML	14
1.4	Where it runs	2	5.4	Forth	14
2	Quickstart	3	5.5	Crossing between them	15
2.1	Getting a Poplog	3	6	Hosting a language of your own	17
2.2	The first session	3	6.1	Plant VM code, don't emit machine code	17
2.3	Running files	4	6.2	Three tiers of effort	18
2.4	Documentation, in the system	4	6.3	Tier 3: registering a subsystem	18
2.5	Editing and long-lived sessions	4	6.4	The other starting point: extend what is there	18
3	Pop-11 fundamentals	5	7	The C interface	19
3.1	Syntax at a glance	5	7.1	The mechanisms, briefly	19
3.2	Words and strings	5	7.2	The shim discipline	19
3.3	Lists, vectors, and the quoting trap	5	7.3	Declaring it: <code>exload</code>	20
3.4	Procedures	5	7.4	Calling it: <code>exacc</code>	20
3.5	The open stack	6	7.5	Building and shipping the shim	21
3.6	Control structures	6	7.6	Images, handles, and when not to bother .	21
3.7	Scope, and <code>dlocal</code>	7	8	Where to go next	23
3.8	The list matcher	7	8.1	The documentation you already have . . .	23
3.9	Objects: <code>Objectclass</code>	8	8.2	Four decades of teaching material	23
4	Worked examples	9	8.3	Reading the source	23
4.1	Log triage: regular expressions, done Poplog's way	9	8.4	Benchmarks, ports, and the state of the project	24
4.2	A web service in pure Pop-11	10	A	Cheat sheet	25
4.3	ELIZA, with the real machinery	11	A.1	Syntax	25
4.4	Graphics, briefly	11	A.2	Blocks — every opener has a closer . . .	25
4.5	Working against a live image	11	A.3	Declarations	25
			A.4	Mishap decoder	26
			A.5	Files, processes, tables	26
			A.6	Regular expressions	26
			A.7	Running things	27

CHAPTER 1

What Poplog is, and why it is unusual

Poplog is not a language. It is a *language toolkit* that happens to ship with five languages already built on it: Pop-11, Prolog, Common Lisp, Standard ML, and — new in this fork — Forth. All five share one virtual machine, one garbage collector, one heap, and one process. You can start in Prolog, drop into Pop-11 to write a predicate, and hand the result back, without leaving the image or crossing a foreign-function boundary.

That design is forty years old and still rare. Most systems that host several languages do it by compiling them all to a bytecode interpreter. Poplog does it by compiling them all, incrementally, to **native machine code**.

1.1 The two-level virtual machine

Poplog’s central idea is a split that lets language implementation and machine porting proceed independently.

- **Above the line**, a language front-end reads source and *plants* instructions for an abstract stack machine — the Poplog VM. It does this by calling about forty ordinary Pop-11 procedures (`sysPUSHQ`, `sysCALL`, `sysPROCEDURE`, ...). A front-end is a Pop-11 program. It contains no assembly and no knowledge of any CPU.
- **Below the line**, a back-end translates planted VM code into real instructions for the host architecture. This is where a port lives: the x86-64, AArch64, RISC-V and ARM32 code generators are each one directory under `pop/src/syscomp/`.

The two halves never meet. A language hosted on the VM inherits every port for free — the Forth subsystem described in Chapter 5 was written once, in Pop-11, and runs natively on Apple Silicon, on a Raspberry Pi 5 and on a StarFive RISC-V board with no per-platform work at all. Conversely, a new port makes every hosted language native on that machine the day it lands.

1.2 Incremental compilation, measured

“Incremental” here means what it says: the compiler is a procedure in the running system, and compiling a definition is an ordinary call. There is no build step, no link step, and no separate compile-time world.

```
vars t0 = systime(), k;  
for k from 1 to 10000 do  
  pop11_compile(stringin('define gen_' sys_>< k sys_><  
                        '(x); x + ' sys_>< k sys_>< ' enddefine;'));  
endfor;  
npr('10000 definitions compiled in ' sys_>< ((systime() - t0) * 10) sys_>< ' ms CPU');  
gen_9999(1) =>
```

```
10000 definitions compiled in 150 ms CPU  
** 10000
```

Fifteen microseconds per definition, each one becoming a real, callable, garbage-collected, *natively compiled* procedure. This is the number that explains most of what follows: when compilation is that cheap, you stop treating it as an event. Redefining a procedure in a live session is not a restart, it is a keystroke.

For calibration, the project’s benchmark suite compares the same workloads against CPython on

identical hardware (see `BENCHMARKS.md` in the source tree for methodology and the full matrix):

Workload	Poplog (x86-64)	Python 3.13	Python 3.10
nfib29 — 1.35M recursive calls	14.5 ms	3.67× slower	8.56×
intloop10M — tight integer loop	61.5 ms	5.60×	6.26×
lists — 1M cons cells, allocate + walk	11.4 ms	7.20×	7.76×
closures1M — 100k closures, 1M calls	22.0 ms	2.13×	3.00×
compile500 — runtime-compile ×500	66.7 ms	0.12×	0.08×

The last row is the honest one, and it is worth reading carefully: Python wins it, because `compile()` produces bytecode while Poplog produces machine code. Poplog is doing strictly more work and still lands within an order of magnitude. On the other rows — calls, loops, allocation, closures — the native code shows.

1.3 The saved image

Because the VM owns the whole heap, the whole heap can be written to a file and read back. `syssave` does exactly that; the resulting `.psv` image contains your data *and* your compiled procedures.

```
$ popsession checkpoint session.psv      # 10,000 procedures live in the heap
2.9M session.psv
$ popsession restore session.psv         # a fresh process, 64 ms including spawn
$ gen_9999(1) =>
** 10000
```

This is how the shipped languages arrive: `clisp.psv`, `prolog.psv` and `pml.psv` are each an image of a Pop-11 system that has already compiled a language on top of itself. It is also how a long-running agent or editor session survives a restart, and why Poplog starts fast — no front-end is ever re-read from source at startup.

What is *not* in the image

Heap state restores; operating-system state does not. Open files, sockets, and pointers into dynamically loaded C libraries are stale after a restore. The discipline is simply to reopen them — Chapter 7 returns to this.

1.4 Where it runs

OS	Architecture	Status
Linux	x86-64	Supported — reference platform
Linux	AArch64	Supported — Raspberry Pi 5, MediaTek Genio 720
macOS	Apple Silicon	Supported — native Mach-O, C ↔ Pop call-backs
Linux	RISC-V RV64GC	Supported — self-hosting on StarFive Vision-Five
Linux	ARM32 (v6/v7)	Supported — long-standing port, Raspberry Pi 1–3
Solaris / FreeBSD	x86 / x86-64	Supported — upstream ports
Windows	x86-64	Not yet ported (WSL2 runs the Linux build)

“Supported” means it builds, self-hosts, and runs all the languages. The first four rows are benchmarked in this fork.

CHAPTER 2

Quickstart

2.1 Getting a Poplog

Three ways in, fastest first.

1. A relocatable binary (about a 2 MB download). This unpacks a ready-built basepop11 plus libraries under `~/.local/share/pop11-skill`, and finishes with a live smoke test:

```
curl -fsSL https://raw.githubusercontent.com/IoTone/poplog/master/tools/install-skill.sh | sh
```

2. From source. Poplog bootstraps from a small seed binary, `corepop`, which is vendored for the ported platforms:

```
git clone https://github.com/IoTone/poplog && cd poplog
./configure && make all
```

`make all` builds the core system, then compiles the Prolog, Common Lisp and Standard ML front-ends and saves each as an image under `target/psv/`. See `INSTALL` for the details.

3. Nix. A flake builds and bootstraps everything from source with no manual seed or toolchain setup:

```
nix run .#pop11          # or .#prolog / .#clisp / .#pml
nix shell .#poplog       # put all five front-ends on $PATH
```

2.2 The first session

```
$ ./poplog pop11
Sussex Poplog (Version 16.019997 ...)
Copyright (c) 1982-1999 University of Sussex. All rights reserved.

Setpop
: 6 * 7 =>
** 42
```

The prompt is `:` and the workhorse is `=>`, which prints whatever the preceding expression left, prefixed with `**`. Two more you will use constantly: `npr(x)` prints `x` and a newline, and `=>` is `=>` with fuller structure printing.

Statements end with a semicolon. An expression followed by `=>` does not need one — `=>` terminates it.

When something goes wrong you get a *mishap*: a diagnostic naming the error, the culprits, and the chain of procedures that was running.

```
: 'three' + 4 =>
;;; MISHAP - NUMBER(S) NEEDED
;;; INVOLVING: 'three' 4
;;; PRINT DOING
;;; DOING      : + pop_setpop_compiler
```

The session survives. Fix the expression and carry on — this is the normal working rhythm, not an exceptional one.

2.3 Running files

A Pop-11 source file is just a sequence of statements, so there is no main:

```
./poplog basepop11 myprog.p      # compile and run, then enter the REPL
./poplog basepop11 myprog.p </dev/null  # ...or run and exit
```

Inside a session, load `'myprog.p'`; does the same thing, and uses `foo`; loads library `foo` from the library search list if it is not already loaded — this is how you reach `lib json`, `lib crypto`, `objectclass`, and the rest.

2.4 Documentation, in the system

Poplog ships its own manual — over 900 files — and it is queryable from the prompt. The three kinds matter:

- `help name` — practical, task-shaped. Start here.
- `ref name` — the exhaustive reference for a primitive or a subsystem.
- `teach name` — tutorials, many of them the original Birmingham AI course material.

```
: help matches
: ref regexp
: teach json
```

The same corpus is published as a website by a Pop-11 program in the tree (`tools/gen-docs.sh`) at <https://iotone.github.io/poplog/>, with an `llms.txt` for AI assistants.

2.5 Editing and long-lived sessions

Because the interesting object is a *running heap*, the tooling in this fork is built to talk to one rather than to a file.

- **VED**, the built-in editor: `ved myfile.p`. ENTER `l1` compiles the current line into the live system, ENTER `lmr` the marked range, ENTER `lcp` the current procedure.
- **LSP** (`tools/pop11-lsp`): diagnostics come from the real compiler with `pop_syntax_only` set, so your file is type-checked by the thing that would compile it, and nothing in it runs. A Neovim plugin ships in `editors/nvim/`.
- **Swank** (`tools/pop11-swank`): the other half — a live session an editor can attach to. Output streams as it is produced, mishaps arrive as data with real frames, and a runaway loop can be interrupted. The Emacs package in `editors/emacs/` is the client; `teach swank` is the walkthrough.
- **MCP** (`tools/pop11-mcp`): the same idea for agents. A persistent, natively compiled session exposed as four tools — `pop11_eval`, `pop11_help`, `pop11_checkpoint`, `pop11_state`.

All three servers sit on one transport, `pop/lib/lib/jsonrpc.p`, itself written in Pop-11.

The working pattern

Start one session and keep it. Define helpers once; call them thereafter. Redefinition is instant and applies to every later call, so a procedure is something you *sculpt* in a live image rather than edit-compile-run. Checkpoint the image when the session has earned something worth keeping.

CHAPTER 3

Pop-11 fundamentals

Pop-11 reads like an Algol-descended imperative language and behaves like a Lisp with a stack machine underneath. It has infix syntax, block keywords that always close (`endif`, `endfor`, `enddefine`), dynamic typing, garbage collection, first-class procedures, and an extensible reader. What makes it distinctive is the *open stack*, and almost everything idiomatic follows from it.

3.1 Syntax at a glance

```
;;; comments run from three semicolons to end of line
vars x = 5;                ;;; a global (permanent) variable
x + 1 -> x;                ;;; ASSIGNMENT POINTS RIGHT: value -> destination
npr('x is ' sys_>< x);     ;;; print a line; sys_>< concatenates anything
x =>                        ;;; the REPL's printer: "** 6"
```

The assignment arrow is the first thing to internalise: `-> x` reads “into x”. It points that way because assignment is a stack operation — the value is already there, and the arrow says where to put it.

3.2 Words and strings

```
'a string'                ;;; single quotes: a mutable vector of characters
"aword"                   ;;; double quotes: a WORD -- an interned symbol, not a string
```

This is the single most common beginner error, and it has a silent cousin. Words are interned: two occurrences of `"apples"` are the same object. Strings are not: two occurrences of `'apples'` are two objects that merely print alike. Poplog’s default hash table, `newproperty`, matches keys by *identity*, so string keys quietly miss:

```
vars tbl = newproperty([], 50, false, true);
3 -> tbl("apples");
tbl("apples") =>          ;;; ** 3      -- words are identity-equal
tbl("pears")  =>          ;;; ** <false> -- the default

vars m = newmapping([], 50, false, true);    ;;; compares keys with =
7 -> m('a string key');
m('a string key') =>      ;;; ** 7
```

Use words as keys, or use `newmapping`. Both are right; silently missing is not.

3.3 Lists, vectors, and the quoting trap

```
[1 2 3]                   ;;; list:    hd(1), tl(1), length(1), l(2)
{1 2 3}                   ;;; vector:  constant-time subscript
```

List brackets *quote* their contents, like Lisp’s `'(...)`. So `[x (n + 1)]` is a list of the literal words `x` `(` `n + 1` `)`. To evaluate, escape with `^` (one item) or `^^` (splice a list):

```
vars n = 4, rest = [d e];
[^n ^(n + 1) c ^^rest] =>    ;;; ** [4 5 c d e]
```

3.4 Procedures

A procedure’s result is whatever it leaves behind; there is no `return` statement, and the last expression is the value.

```
define double(n);
  n * 2
enddefine;
double(21) =>                ;;; ** 42
```

A procedure may leave *several* results, which the caller collects in reverse order:

```
define divmod(a, b);
  a div b, a mod b
enddefine;
vars q, r;
divmod(17, 5) -> r -> q;
npr('17 = ' sys_>< q sys_>< '*5 + ' sys_>< r);    ;;; 17 = 3*5 + 2
```

Closures are made by partial application with `% . . %`:

```
define mult(a, b); a * b enddefine;
vars triple = mult(% 3 %);
triple(14) =>                ;;; ** 42
```

And every accessor has an *updater* — the procedure that runs when it appears on the left of an arrow. This is why `l(2)` both reads and writes, and why you can give your own procedures assignable behaviour:

```
vars l = [a b c d];
l(2) =>                        ;;; ** b
"BEE" -> l(2);
l =>                          ;;; ** [a BEE c d]
```

3.5 The open stack

There is one user-visible stack, and it is not hidden by the calling convention. A procedure can leave zero, one, or a hundred results, and the number need not be known until it runs.

```
define upto(n);
  lvars i;
  for i from 1 to n do i endfor    ;;; leaves n results on the stack
enddefine;

[% upto(5) %] =>                ;;; ** [1 2 3 4 5]
{% upto(5) %} =>                ;;; ** {1 2 3 4 5}
```

`[% . . . %]` means “run this, and make a list of everything it left”. That one construct replaces most of what other languages need generators, varargs, spread operators and comprehensions for. It also makes Forth nearly free to host (Chapter 6): Forth’s data stack *is* this stack.

Why this matters for performance

Because results are passed on the stack rather than boxed into tuples, multiple return values cost nothing, and neither does a procedure that sometimes returns nothing. The `nfib` and `closures1M` benchmark rows in Chapter 1 are measuring this calling convention.

3.6 Control structures

Every opener has a matching closer.

```
if x > 3 then npr('big') elseif x = 3 then npr('exact') else npr('small') endif;
unless ready then wait() endunless;

for i from 1 to 10 do ... endfor;
```

```

for item in list do ... endfor;
while cond do ... endwhile;
repeat ... quitif(done); ... endrepeat;

define classify(n);
  switchon n
    case < 0 then 'negative'
    case = 0 then 'zero'
    else 'positive'
  endswitchon
enddefine;
classify(-5) =>          ;;; ** negative

```

3.7 Scope, and dlocal

vars declares a permanent (global) identifier; lvars declares a lexical local inside a procedure — prefer lvars. constant and lconstant are their immutable counterparts.

dlocal is the one with no close equivalent elsewhere: it gives a variable a new value for the *dynamic extent* of a procedure call, and restores the old value on exit — whether the procedure returns normally, jumps out, or is unwound by a mishap.

```

vars indent = '';
define show(x);
  dlocal indent = indent >< ' ';
  npr(indent sys_>< x);
  if x > 1 then show(x - 1) endif;
enddefine;
show(3);

```

```

3
  2
    1

```

After show returns, indent is empty again. The same mechanism is how Poplog rebinds output streams, the current character repeater, and the compiler's own state safely — it is Pop-11's answer to try. . . finally, and it is declarative.

3.8 The list matcher

Pop-11's pattern matcher works on lists and is built into the language with the matches operator. ?x binds one item, ??x binds a (possibly empty) segment, = matches one item without binding.

```

vars db = [[tom likes cats] [ann likes dogs] [tom likes jazz]];
vars item, what;          ;;; pattern variables must be permanent (vars)
for item in db do
  if item matches [tom likes ?what] then
    npr('tom likes ' sys_>< what)
  endif
endfor;

```

```

tom likes cats
tom likes jazz

```

A trap worth naming

The matcher assigns through the *permanent* identifier, so pattern variables declared with lvars are silently not filled in — you get <undef>. Declare them vars.

This matcher, plus a dozen lines of ranking and reassembly, is the whole of ELIZA; `examples/eliza.p` in the tree is a faithful implementation of Weizenbaum's original, script file and all.

3.9 Objects: Objectclass

Objectclass is Pop-11's CLOS-like object system: classes with slots, and generic procedures with methods dispatched on argument class.

```
uses objectclass;

define :class Account;
  slot acct_name = 'anon';
  slot acct_balance = 0;
enddefine;

define :method deposit(a:Account, n);
  acct_balance(a) + n -> acct_balance(a);
  a
enddefine;

define :method print_instance(a:Account);
  printf('<Account %p: %p>\n', [% acct_name(a), acct_balance(a) %]);
enddefine;

vars acc = newAccount();
'ada' -> acct_name(acc);
deposit(acc, 100);
deposit(acc, 42);
print_instance(acc);          ;; <Account ada: 142>
```

Slots come with accessors *and* updaters for free, which is why `acct_balance(a)` works on both sides of the arrow.

CHAPTER 4

Worked examples

Four programs that do real work. Each was run against the tree while this chapter was written; the transcripts are the actual output.

4.1 Log triage: regular expressions, done Poplog's way

Poplog ships a full regexp engine (ref `regexp`) — but with inverted syntax, and this catches everyone exactly once. **The escape character is @, not backslash, and . * [] ^ \$ are literal by default.** A pattern pasted in from any other engine compiles without complaint and then matches almost nothing.

PCRE	Poplog	PCRE	Poplog
.	@.	x?	x@{0,1@}
*	@*	x{m,n}	x@{m,n@}
[abc]	@[abc@]	^ \$	@^ @ \$
x+	x@{1,@}	(x)	@(x@)

There is no alternation (`|`) and no lookahead: compile one pattern per branch. In exchange, `regexp_compile` hands back a *compiled procedure* you define once and then call at native speed.

```
;;; triage.p -- count log levels and find the slowest reported query
uses regexp;

vars err, match_ms;
regexp_compile('@[0-9@]@[1,@] ms') -> (err, match_ms);
if err then mishap(err, 0, 'bad pattern') endif;

define triage(path) -> (counts, slowest);
  lvars dev, rep, line, lev, i, n, ms;
  newproperty([], 8, 0, true) -> counts;      ;; default 0, so += just works
  0 -> slowest;
  sysopen(path, 0, "line") -> dev;
  line_repeater(dev, inits(4096)) -> rep;      ;; size the buffer generously
  repeat
    rep() -> line;
    quitif(line == termin);
    for lev in [INFO WARN ERROR] do
      if issubstring(lev sys_<< ' ', 1, line) then
        counts(lev) + 1 -> counts(lev)
      endif
    endfor;
    match_ms(1, line, false, false) -> (i, n);
    if i then
      strnumber(substring(i, n - 3, line)) -> ms;
      if ms and ms > slowest then ms -> slowest endif
    endif;
  endrepeat;
enddefine;
```

Against a seven-line sample log:

```
INFO: 2
WARN: 2
ERROR: 3
slowest query: 1420 ms
```

Two idioms are doing quiet work here. `newproperty([], 8, 0, true)` gives a table whose *default* is 0, so incrementing an unseen key needs no initialisation branch. And `line_repeater` returns a procedure that yields one line per call and `termin` at end of file — the standard Poplog shape for a stream, and the same shape `sys_obey_linerep` uses for the output of a shell command.

4.2 A web service in pure Pop-11

`lib http_server` and `lib json` compose into a service with routing, query and body access, and the usual safety nets — an unknown route answers 404, and a mishap inside a handler answers 500 without taking the server down. The whole program is the handler.

```
uses http_server;
uses json;

define handler(req) -> resp;
  lvars path = req('path'), obj;
  if path = '/hello' then
    http_text('hello from Pop-11\n') -> resp;
  elseif path = '/greet' then
    newmapping([], 4, false, true) -> obj;
    req('query') -> obj('query');
    req('method') -> obj('method');
    http_json(obj) -> resp;
  elseif path = '/info' then
    newmapping([], 4, false, true) -> obj;
    'poplog' -> obj('server');
    pop_internal_version -> obj('version');
    http_json(obj) -> resp;
  elseif path = '/boom' then
    mishap(0, 'deliberate handler failure');    ;;; server answers 500
  else
    false -> resp;                             ;;; server answers 404
  endif;
enddefine;

http_serve_n(8099, handler, false);
```

```
$ curl -s localhost:8099/hello
hello from Pop-11
$ curl -s 'localhost:8099/greet?name=poplog'
{"method":"GET","query":"name=poplog"}
$ curl -s localhost:8099/info
{"version":160200,"server":"poplog"}
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8099/nope
404
```

`lib json` maps JSON onto Pop-11 directly: an object becomes a `newmapping` property, an array a vector, a string a string of UTF-8 bytes, and null the distinguished word `json_null` (kept apart from `false`, which is JSON `false`).

```
uses json;
vars v = json_parse('{"name": "poplog", "ports": [64, 32]}');
v('name') => ;;; ** poplog
v('ports')(1) => ;;; ** 64
json_generate(v) => ;;; ** {"ports":[64,32],"name":"poplog"}
```

Object member order is not preserved — properties are hash tables, and RFC 8259 promises nothing about ordering. The file is `examples/http_hello.p`; teach `json` walks through how `lib json` itself was built, bug included.

4.3 ELIZA, with the real machinery

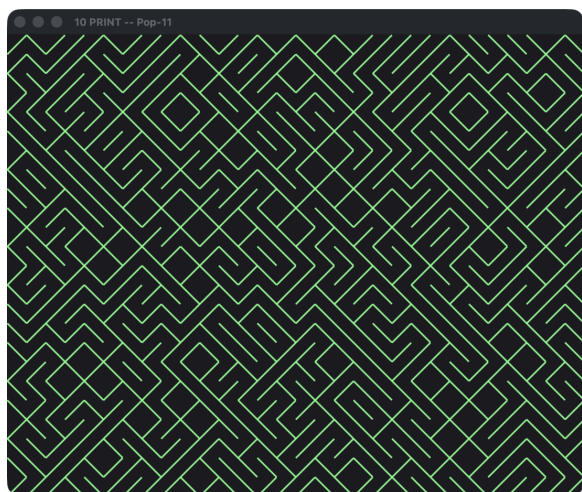
examples/eliza.p implements Weizenbaum’s 1966 DOCTOR properly — keyword ranking, decomposition rules with wildcards and synonym classes, reassembly templates cycled round-robin, pronoun reflection, goto equivalence, and the MEMORY mechanism that stows what you said about “my ...” and brings it back later. The script lives in a separate data file, doctor.txt, exactly as Weizenbaum separated script from engine.

The reason it fits in a couple of dozen lines of matching logic is §3.8’s matcher: a decomposition rule *is* a Pop-11 list pattern.

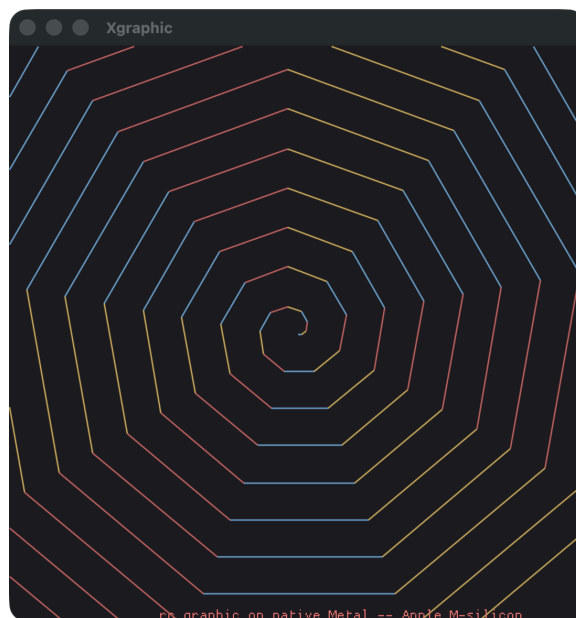
```
How do you do. Please tell me your problem.
Well, my boyfriend made me come here.
Your boyfriend made you come here?
He says I am depressed much of the time.
I am sorry to hear that you are depressed.
My mother takes care of me.
Tell me more about your family.
It is raining today.          <- no keyword: a stored memory returns
Lets discuss further why your mother takes care of you.
```

4.4 Graphics, briefly

Poplog’s graphics were historically tied to X. This fork adds an optional native backend built on Dear ImGui, chosen at build time with `./configure --experimental-graphics`: Metal + Cocoa on macOS, or SDL3 + OpenGL3 on macOS or Linux — and SDL3 picks its display transport at run time, so Wayland, X11, KMS/DRM and headless software rendering all work from one build. The classic `rc_graphic` turtle library and `rc_mouse` are ported onto it, so existing Pop-11 graphics code runs unchanged.



examples/tenprint.p — the 10 PRINT maze



rc_graphic turtle on the Metal backend

Graphics are strictly opt-in: the default build, and `nix build .#poplog`, are console-only.

4.5 Working against a live image

The last example is not a program but a habit. Because compiling is microseconds and saving the heap is milliseconds, the natural unit of work is a *session* that accumulates state, with snapshots

at the points worth returning to.

```
$ popsession start --name triage          # boot a session (~50 ms)
$ popsession send  --name triage -f triage.p      # define the helpers once
$ popsession send  --name triage -c "trriage('app.log') =>"    # ~1 ms per call
$ popsession checkpoint --name triage scan-v1.psv # freeze heap + procedures
$ popsession restore  --name triage scan-v1.psv  # a new process, fully loaded
```

The image contains the compiled procedures, so restoring is not re-loading:

```
$ ls -lh session.psv
2.9M session.psv          # a heap holding 10,000 compiled procedures
$ time popsession restore session.psv
0.064 total                # including process spawn
$ gen_9999(1) =>
** 10000
```

Two cautions, both from §1.4. A restore rolls the *whole* session back — anything defined after the snapshot is gone, so re-checkpoint after adding to a restored session. And handles into the operating system or into C libraries go stale across a restore; reopen them rather than trusting them.

CHAPTER 5

The hosted languages

Poplog's front-ends are not bindings, bridges or embeddings. Each is a compiler written in Pop-11 that plants code for the same VM, and the result lives in the same heap as everything else. Switching language switches the *reader*; it does not switch worlds.

```
dkords — dkords@Mac — — -zsh — 120x37
Sussex Poplog (Version 16.019997 Wed Jun 10 21:42:09 PDT 2026)
Copyright (c) 1982-1999 University of Sussex. All rights reserved.

Setpop
: 6 * 7 =>
** 42
: define twice(x): x = 2 enddefine;
: maplist([1 2 3 4 5], twice) =>
** [2 4 6 8 10]
: length(the quick brown fox) =>
** 5
: '0
- - []
```

Pop-11 — the core language

```
dkords — dkords@Mac — — -zsh — 120x37
Sussex Poplog (Version 16.019997 Wed Jun 10 21:42:09 PDT 2026)
Copyright (c) 1982-1999 University of Sussex. All rights reserved.
Prolog (Version 3.2)

Setprolog
?- X is 6 * 7, write('6 * 7 = '), write(X), nl.
6 * 7 = 42
X = 42 ?
yes

?- [First | Rest] = [red, green, blue], write('first = '), write(First), nl.
first = red
First = red
Rest = [green, blue] ?
yes

?- (7 > 3 => write('7 > 3 holds')) ; write(no), nl.
7 > 3 holds
yes

?- '0
- - []
```

Prolog — Edinburgh syntax

```
dkords — dkords@Mac — — -zsh — 120x37
Sussex Poplog (Version 16.019997 Wed Jun 10 21:42:09 PDT 2026)
Copyright (c) 1982-1999 University of Sussex. All rights reserved.
Common Lisp (Version 2.0)

Setlisp
== (* 5 7)
42
== (mapcar (function (lambda (x) (* x x))) '(1 2 3 4 5))
(1 4 9 16 25)
== (defun fact (n) (if (= n 0) 1 (* n (fact (- n 1)))))
FACT
== (fact 10)
3628800
== '0
- - []
```

Common Lisp — CLTL2

```
dkords — dkords@Mac — — -zsh — 120x37
Sussex Poplog (Version 16.019997 Wed Jun 10 21:42:09 PDT 2026)
Copyright (c) 1982-1999 University of Sussex. All rights reserved.
Standard ML (Version 2.1)

Setml
- print "Standard ML on Poplog:\n";
"Standard ML on Poplog:\n"
- val it = "Standard ML on Poplog:\n" : string
- fun fact n = if n = 0 then 1 else n * fact (n-1);
- val fact = fn : int -> int
- print (makestring (fact 22) ^ "\n");
"679001600\n"
- val it = "679001600\n" : string
- map (fn x => x + 1) [10, 20, 30];
val it = [11, 21, 31] : int list
- print "done\n";
"done\n"
- val it = "done\n" : string
- '0
- - []
```

Standard ML — type inference

Four front-ends, one engine, one banner — captured on the Apple Silicon build. Each image is the same basepop11 with a different language compiled into it.

5.1 Prolog

Edinburgh-syntax Prolog, implemented as 51 Pop-11 files under `pop/plog/src/` — reader, term representation, unification, clause store and control. Because it is open Pop-11 rather than a sealed engine, extending it with new built-ins is an afternoon's work, not a fork.

```
parent(tom, ann).
parent(tom, liz).
parent(ann, kim).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).
```

```
$ ./poplog basepop11 -target/psv/prolog.psv
?- ['fam.pl'].
fam.pl consulted
yes
?- grandparent(tom, W).
W = kim
?- ancestor(tom, W).
W = ann W = liz W = kim
?- parent(P, kim).
P = ann
```

A small surprise

Poplog's Prolog is deliberately lean: list predicates such as `member/2` and `append/3` are *not* built in. Several library files under `pop/plog/lib/` define them, or you write the two clauses yourself — which, for a system whose point is that you can see and change the engine, is in keeping.

5.2 Common Lisp

Compatible with most of CLTL2. It is a genuine Common Lisp — packages, macros, `format`, the condition system — compiled to native code by the same back-end as everything else.

```
(defun sq (x) (* x x))
(print (sq 9))
(print (mapcar (function sq) (list 1 2 3)))
```

```
SQ
81
(1 4 9)
```

Remember that the reader upcases: `sq` and `SQ` are the same symbol, and `#`, `.`, `,`, `@` and `:` are reader macros. `help poplisp` is a translation table for Pop-11 programmers arriving in Lisp, and `help poplisp` documents the mixed-language interface.

5.3 Standard ML

A full Standard ML with Hindley–Milner type inference, and the strongest demonstration that the VM is language-agnostic: static typing, pattern-matching function clauses and polymorphism all sit on the same dynamically typed heap.

```
fun sq (x:int) = x * x;
map sq [1,2,3];
fun fact 0 = 1 | fact n = n * fact(n-1);
fact 10;
```

```
val sq = fn : int -> int
val it = [1, 4, 9] : int list
val fact = fn : int -> int
val it = 3628800 : int
```

The type-checker is where ML's surprises live rather than the runtime: an unannotated `fun sq x = x * x` is rejected, because `*` is overloaded and there is nothing to resolve it against.

5.4 Forth

Those three, with Pop-11 itself, are classic Poplog. This fork adds a fifth, and it is the shortest route to understanding why hosting on the VM is cheap: **Forth's data stack is the Poplog user stack**. A Forth primitive is an ordinary open-stack Pop-11 procedure — `+` is `define f_plus(x,y); x+y enddefine` — and no data structure is interposed at all.

Colon definitions are not threaded code. `: name . . . ;` is transpiled to a Pop-11 procedure and run through the incremental compiler, so a Forth word is a real native procedure; `if/else/then`, `begin/until`, `begin/while/repeat`, `do/loop`, `recurse` and `exit` map onto Pop-11 constructs at compile time.

```
: sq dup * ;
9 sq .
: fib dup 2 < if drop 1 else dup 1 - recurse swap 2 - recurse + then ;
```

```
20 fib .
: stars 0 do 42 emit loop ;
10 stars cr
variable acc 5 acc ! acc @ .
```

```
$ tools/forth.sh
Poplog Forth. words / testbench / bye
81 ok
10946 ok
*****
ok
5 ok
forth> testbench
21 passed, 0 failed ALL OK (0.273 ms)
```

The core covers arithmetic, about forty stack, comparison, bitwise and I/O words, native colon definitions, the control words above, counted loops with `i/j/leave`, `variable/constant/@/!`, a return stack (`>r r@ r>`), and string literals. It is younger and leaner than the mature four — no `+loop`, `create does>` or `value yet`, and it is case-sensitive. Because it is pure Pop-11 it runs on every platform Poplog does; the built-in testbench is 21/21 on both macOS arm64 and RISC-V. Implementation: `pop/forth/src/forth.p`, 523 lines.

5.5 Crossing between them

All five share one image, so crossing is a change of reader, not a marshalling boundary.

From any language back to Pop-11, type `pop11`:

```
$ ./poplog basepop11 -target/psv/clisp.psv
(print (* 6 7))
42
pop11
: [a b c] <> [d] =>
** [a b c d]
```

```
$ ./poplog basepop11 +target/psv/pml.psv
val x = 6 * 7;
val x = 42 : int
pop11
: [a b c] <> [d] =>
** [a b c d]
```

From Pop-11 into Prolog, the syntax word `?-` runs a goal inline, against the same clause store:

```
: vars greeting = [hello there];
: ?- write(from_prolog), nl.
from_prolog
yes
: rev(greeting) =>
** [there hello]
```

Defining Prolog predicates in Pop-11 is a documented form (`help define_prolog`) — useful when a predicate wants a data structure or an I/O facility that is easier to reach from Pop-11:

```
define :prolog hello/0(contn);
  lvars contn;
  printf('Hello world\n');
  chain(contn);          ;; call the continuation = succeed
enddefine;
```

```
?- hello.
```

```
Hello world  
yes
```

The continuation-passing shape is not incidental: it is how Poplog’s Prolog represents “and then carry on solving”, so a Pop-11 predicate that calls `chain(contn)` succeeds once, and one that calls it in a loop is a generator with backtracking.

Which image has which language

Each shipped image contains the languages compiled into it. `prolog.psv` has Pop-11 and Prolog; `clisp.psv` has Pop-11 and Common Lisp; `pml.psv` has Pop-11 and ML. Forth is pure library code — `uses forth` works anywhere. To get several at once, build an image that loads them, exactly as `make` builds these.

CHAPTER 6

Hosting a language of your own

Chapter 5’s front-ends are not privileged. The API they target is public, and a new language is an ordinary Pop-11 program. This chapter is the recipe; HOSTING-LANGUAGES.md in the tree is the longer version, and ref vmcode is the specification.

6.1 Plant VM code, don’t emit machine code

You write a **reader**, a **parser**, and then calls to about forty **VM-builder** procedures that plant abstract instructions. The back-end turns those into native code for whatever CPU you are on.

To plant...	call
a literal constant	sysPUSHQ(item)
the value of a variable	sysPUSH(word)
a store into a variable	sysPOP(word)
a call of a named procedure	sysCALL(word)
a call of a literal procedure	sysCALLQ(item)
start / end of a procedure	sysPROCEDURE(props, nargs) ... sysENDPROCEDURE()
locals, dynamic locals	sysLVARs, sysLOCAL, sysNEW_LVAR
control flow	sysLABEL, sysGOTO, sysIFSO, sysIFNOT, sysGO_ON
a permanent assignment	sysPASSIGN(item, token)
a syntax word / macro	sysSYNTAX

The whole model fits in six lines. Here we build, at run time, the equivalent of `define add1(x); x fi_+ 1 enddefine`:

```
define build_add1() -> p;
  sysPROCEDURE(false, 1);      ;; open a 1-argument procedure
  sysPUSHQ(1);                 ;; push literal 1 -> stack: x 1
  sysCALL("fi_+");             ;; fast integer add -> stack: x+1
  sysENDPROCEDURE() -> p;      ;; p is now a real compiled procedure
enddefine;

vars add1 = build_add1();
isprocedure(add1) =>           ;; ** <true>
pdnargs(add1)    =>           ;; ** 1
add1(41)         =>           ;; ** 42
```

Execute level

Planting must happen *while something is running*, not at the top level of a file being compiled — at the top level you get `sysEXECUTE: NOT AT EXECUTE LEVEL`. Wrap the plant sequence in a procedure and call it, as above.

That is the entire trick. A compiler for your language is a tree walk that emits the right `sys. . .` calls. What you get for free, without writing a line of assembly: native code generation on every port, the garbage collector, the open stack, closures, recursion, the incremental compiler, the external interface, the REPL, and saved images.

For the reader, you can reuse Pop-11’s `itemiser` (`incharitem` over `proglis`t, documented in `ref popcompile`) with your own character table — for a whitespace-delimited language like Forth it needs almost no configuration — or write your own, as Prolog does with its operator-precedence reader.

6.2 Three tiers of effort

Tier	What you get	Effort
1. Embedded interpreter	A plain Pop-11 program: a dictionary and an eval loop. No VM compilation.	days
2. VM-compiling front-end	Parse, then plant — genuinely native definitions, closures, recursion, full interop. <i>The Poplog way</i> .	1–3 weeks
3. Full subsystem	Tier 2 plus registration: its own REPL, prompt, banner, file extension and saved image.	+1–3 days

The tiers stack. Build tier 1 as a throwaway to pin the semantics with no compiler in the way, replace its core with planting, then wrap the subsystem around it.

6.3 Tier 3: registering a subsystem

A subsystem is a record in `sys_subsystem_table` with a name, a file extension, a prompt, a title, search lists, and an eight-slot procedure vector in a fixed order (`ref subsystem`):

```
constant myforth_subsystem_procedures =
  {% forth_compile,      ;; SS_COMPILER  <- the only substantial one
    identfn,             ;; SS_RESET    (between top-level reads)
    identfn,             ;; SS_SETUP    (on startup)
    Forth_banner,        ;; SS_BANNER
    identfn,             ;; SS_INITCOMP (compile a standard prelude)
    erase,               ;; SS_POPARG1  (handle a command-line argument)
    identfn,             ;; SS_VEDSETUP (editor; no-op if you skip Ved)
    identfn,             ;; SS_XSETUP   (X; no-op if you skip graphics)
  %};
```

Only `SS_COMPILER` is real work — it is your tier-2 compiler. The rest can start as `identfn/erase` stubs. One macro then makes the language reachable by name from a Pop-11 prompt:

```
define global macro forth;
  "forth" -> sys_compiler_subsystem(`c`);
enddefine;
```

Finally, build an image the way the shipped languages are built — a `mkimage` invocation that loads your sources and installs `mylang.psv` — and you have `poplog mylang`, live subsystem switching, and your language on every platform Poplog runs on. The four existing subsystems under `pop/lisp/`, `pop/pm1/`, `pop/plog/` and `pop/forth/` are the templates; each is a `*_subsystem_procedures` file, a compiler, and a Makefile rule.

6.4 The other starting point: extend what is there

Not every new language needs to be new. The shipped Prolog is 51 files of ordinary Pop-11, so a specialised variant has an enormous head start: reuse the term reader, unification and clause store, and replace only the *control* — a different search order, tabling, constraint propagation, a custom resolution rule. Adding built-ins and operators to the existing engine is hours; swapping the control regime is days to weeks, scaling with how much of the engine you replace.

CHAPTER 7

The C interface

Poplog has had a foreign-function interface since 1987, and it is unusually deep: external code is loaded into the *running* image by Poplog’s own dynamic linker, declared with a syntax form, and called with argument conversion that understands Poplog’s data representation. This chapter shows the path this fork actually uses in production libraries, then names the alternatives and the traps.

7.1 The mechanisms, briefly

<code>external_do_load</code>	The primitive: a mark item, a list of object files or shared libraries, and a list of symbol specifications. Procedural.
<code>exload</code>	The declarative syntax form over it — name the library, name the functions, give their types. Start here.
<code>exacc</code>	The syntax form that <i>calls</i> an external function or accesses external data, converting arguments and results.
<code>cons_access</code>	Builds an accessor procedure for a field of an external structure.
<code>lib external, lib newexternal</code>	Older higher-level layers; see <code>help external</code> and <code>help newexternal</code> .

The reference material is `ref external` (mechanisms and data representation), `ref defstruct` (structures) and `ref external_data`.

7.2 The shim discipline

The libraries in this fork — `lib crypto` over OpenSSL, `popcurl` over `libcurl`, `poppcrc` over PCRE — all go through a small C shim rather than binding the library’s own API directly. That is a deliberate rule, and it is worth stating plainly:

Bind a shim, not the library

Keep the `exload` surface to a handful of **non-variadic** functions with **explicit lengths** and **caller-allocated buffers**, and keep object lifecycle entirely on the C side. Variadic calls through the FFI are hazardous — notably on Apple `arm64`, where the variadic calling convention differs from the regular one — and library handle lifetimes are exactly what a garbage collector should not be guessing about.

A shim is usually fifty lines. Here are two of the seven functions behind `lib crypto`:

```
/* pop/extern/popcrypto/popcrypto_shim.c */
#include <openssl/evp.h>
#include <openssl/hmac.h>

/* digest of data[0..len) with a named algorithm; writes raw bytes to
   out, returns the digest length, or -1 */
int pcr_digest(const char *alg, const void *data, long len,
              unsigned char *out)
{
    const EVP_MD *md = EVP_get_digestbyname(alg);
    unsigned int n = 0;
    if (!md) return -1;
    if (!EVP_Digest(data, (size_t)len, out, &n, md, NULL)) return -1;
    return (int)n;
}
```

```
int pcr_random(unsigned char *buf, long n)
{
    return RAND_bytes(buf, (int)n) == 1 ? 0 : -1;
}
```

Note the shape: no allocation crosses the boundary, every buffer has an explicit length, and every failure is a negative return rather than an exception or an `errno` the caller must remember to read.

7.3 Declaring it: `exload`

```
uses crypto;    ;;; ...is, inside, this:

exload popcrypto
  #_IF sys_file_exists(sysfileok('$usepop/pop/extern/popcrypto/popcrypto.dylib'))
  ['$usepop/pop/extern/popcrypto/popcrypto.dylib']
  #_ELSE
  ['$usepop/pop/extern/popcrypto/popcrypto.so']
  #_ENDIF
(language C)
  lconstant
    pcr_digest(alg, data, len, out)          :int,
    pcr_hmac(alg, key, keylen, data, len, out) :int,
    pcr_random(buf, n)                       :int,
    pcr_pbkdf2(alg, pass, passlen, salt, saltlen, iters, out, outlen) :int,
    pcr_version()                           :exptr,
    ;;; ...and pcr_gcm_encrypt / pcr_gcm_decrypt
  ;
endexload;
```

Three things are happening. `popcrypto` is the *mark item* — the tag under which this set of symbols is loaded and can be unloaded. `#_IF` is a compile-time conditional, so one source file picks the right shared-library extension on macOS and Linux. And each declaration gives the argument names and the *result* type: `:int` for a C int, `:exptr` for a pointer (Poplog wraps it in an external-pointer record; `exacc_ntstring` pulls a C string out of one). The full type vocabulary — `:byte`, `:short`, `:int`, `:long`, `:sfloat`, `:dfloat`, pointers, arrays and structures — is in `ref external`.

7.4 Calling it: `exacc`

```
lconstant OUTMAX = 64;    ;;; SHA-512 is the widest digest

define crypto_digest(alg, data) -> raw;
  lvars n, out = inits(OUTMAX);    ;;; caller allocates the buffer
  unless isstring(alg) and isstring(data) then
    mishap(alg, data, 2, 'crypto_digest: two strings needed')
  endunless;
  exacc pcr_digest(cstr(alg), data, length(data), out) -> n;
  if n < 0 then
    mishap(alg, 1, 'crypto_digest: unknown algorithm')
  endif;
  substring(1, n, out) -> raw;
enddefine;
```

```
: uses crypto;
: crypto_digest_hex('sha256', 'hello poplog') =>
** 8b05ad6e876a9ce82062d33918c82d6b9bfac5b0f1cb575bb4dc8cd70762c6ae
: crypto_hmac_hex('sha256', 'key', 'message') =>
** 6e9ef29b75fffc5b7abae527d58fdadb2fe42e7219011976917343065f58ed4a
: length(crypto_random(16)) =>
** 16
```

```
: crypto_version() =>
** OpenSSL 3.6.3 9 Jun 2026
```

7.4.1 Two traps that bite immediately

Pop-11 strings are not null-terminated. They are counted byte vectors, so a string passed where C expects `char *` must be terminated explicitly. `lib crypto` does it in one line:

```
define lconstant cstr(s);
  s <> consstring(0, 1)      ;;; append a NUL
enddefine;
```

Data arguments do not need this — they are passed with an explicit length, which is the better interface anyway.

Packed fields versus full fields. Poplog integers and floats are tagged in ordinary records and vectors. Passed as direct arguments they are converted for you; passed *inside* a structure they are not, and the C side sees encoded nonsense. An integer array destined for C must therefore be built with a packed representation (`intvec` and friends), not `newarray`. Strings made with `inits(n)` are packed byte vectors, which is exactly why they serve as C buffers above.

7.5 Building and shipping the shim

The shim is built by a small script, not by the main `make`, so a Poplog without OpenSSL still builds:

```
tools/build-popcrypto.sh      # -> pop/extern/popcrypto/popcrypto.{so,dylib}
```

```
if command -v pkg-config >/dev/null 2>&1 && pkg-config --exists libcrypto; then
  flags="$(pkg-config --cflags --libs libcrypto)"
else
  flags="-lcrypto"
fi
cc -O2 -shared -fPIC -o "$lib" "$src" $flags
```

And the library fails loudly at load time if the shim is missing, rather than mysteriously at the first call:

```
unless sys_file_exists(sysfileok('$usepop/pop/extern/popcrypto/popcrypto.so'))
or   sys_file_exists(sysfileok('$usepop/pop/extern/popcrypto/popcrypto.dylib'))
then
  mishap(0, 'lib crypto: shim not built -- run tools/build-popcrypto.sh first')
endunless;
```

Both `lib json` and its FFI-based sibling `lib crypto` are validated on x86-64, AArch64 (Raspberry Pi 5) and RISC-V; `tools/test-crypto.sh` is the acceptance suite and `tools/test-remote.sh` host reruns it anywhere with a built tree. That matters more than usual for FFI code, because calling conventions are exactly what differs between ports — the RISC-V float ABI, for instance, was found and fixed by running these suites.

7.6 Images, handles, and when not to bother

External state does not survive `sys-save`. Library handles, open connections and `exptr` values are stale after a restore — the right shape for a binding is to *mishap cleanly* on a stale handle rather than crash, and the fix is always to reopen. Procedures, by contrast, survive fine, so a restored image is ready to work as soon as you reconnect its resources.

Finally: the FFI is not always the answer. Poplog can stream the output of a shell command as a line repeater, and for a one-off that is often enough:

```
vars r = sys_obey_linerep('set +e; curl -s https://example.com/api | jq -r ".id"');
```

Note `set +e`: the command line runs under `errexit`, so without it the first failing command aborts the whole pipeline *silently* — empty output and no error.

The trade-off against the FFI is a process per call, and it is not subtle: on the machine this chapter was written on, 100 round trips through `sys_obey_linerep('echo hi')` took 284ms of wall time — about 2.8ms each — against an unmeasurable zero for 100 in-image calls. Use a shim when the call sits in a loop; use the pipe when it does not.

CHAPTER 8

Where to go next

8.1 The documentation you already have

The in-tree corpus is over 900 files and it is the primary source for everything in this book. From any prompt:

```
: help matches      ;;; practical, task-shaped
: ref vmcode        ;;; the exhaustive reference -- and the VM spec
: teach json        ;;; tutorials, including how lib json was built
```

The same material is published at <https://iotone.github.io/poplog/>, regenerated on every push by a Pop-11 program; `llms.txt` there is for AI assistants. The handful of entries worth reading early: `help external` and `ref external` (Chapter 7), `ref vmcode` and `ref subsystem` (Chapter 6), `ref regexp`, `help json`, `help crypto`, and `teach swank`.

8.2 Four decades of teaching material

Poplog was the vehicle for a great deal of university AI teaching, and most of it survives. The tree does not vendor any of it; one script fetches it from public archives and wires it into `help/teach` inside the system:

```
tools/fetch-learning.sh list      # what's available
tools/fetch-learning.sh --all     # about 30 MB into learn/
```

<code>hidden-gems</code>	Small idiomatic demos of what makes Pop-11 distinctive: the open stack, updaters, dynamic lists, coroutines, <code>dlocal</code> , the GC, extensible syntax.
<code>bhamteach</code>	The Birmingham introductory-AI course — pattern-matching chatbots, grammars and parsing, story generation, recursion exercises.
<code>packages</code>	The classic package tree: <code>SimAgent</code> and <code>Poprulebase</code> , <code>popvision</code> , <code>rclib</code> , neural nets, Braitenberg vehicles.
<code>paradigms</code>	A UMASS programming-language-paradigms course taught in Pop-11 (1997–2000).
<code>primer</code>	The Pop-11 Primer — the standard text for experienced programmers.
<code>examples</code>	Curated teaching examples: <code>Othello</code> , <code>Life</code> , <code>robolang</code> .

Upstream licences apply per pack; each records its origin in `learn/pack/.source`.

8.3 Reading the source

Poplog is written mostly in itself, which makes it unusually approachable.

- `pop/src/` — the core: `vm_plant.p` and `vm_control.p` are the VM builder from Chapter 6; `syscomp/` holds one directory per CPU back-end.
- `pop/lib/lib/` — the library, and the best source of idiomatic Pop-11 at length. `json.p`, `crypto.p`, `http_server.p`, `jsonrpc.p`, `swank.p` and `forth.p` are all readable in an afternoon each.
- `pop/plog/`, `pop/lisp/`, `pop/pml/`, `pop/forth/` — the four front-ends, i.e. four worked answers to Chapter 6.
- `examples/` — runnable programs: the HTTP service, ELIZA, a Z-machine interpreter, graphics demos.

8.4 Benchmarks, ports, and the state of the project

BENCHMARKS.md carries the full cross-platform, cross-language matrix with its methodology; the numbers quoted in Chapter 1 are drawn from it. The PORTING-*.md files are the working notes from the AArch64, Apple Silicon and RISC-V ports — including the bugs, which is the part usually missing from porting documentation and the part most useful to the next porter. Windows is the one platform still marked TODO.

If you take one idea away

It is that a compiler cheap enough to call in a loop changes what a programming environment can be. Everything distinctive here — live redefinition, saved images, five languages in one heap, an editor that talks to a running process, an agent that keeps a session warm across tool calls — falls out of fifteen microseconds per definition.

APPENDIX A

Cheat sheet

A.1 Syntax

;;; text	comment to end of line
value -> x	assignment; the arrow points at the destination
expr =>	print the result(s), prefixed **
expr ==>	as => but prints structure in full
npr(x)	print x and a newline
a sys_>< b	concatenate anything into a string
a >< b	concatenate two strings
'text'	string (mutable, counted, <i>not</i> NUL-terminated)
"word"	word — an interned symbol
[a b c]	list; contents are <i>quoted</i>
[^x ^(e) ^^xs]	... with x, e evaluated and xs spliced
{1 2 3}	vector
[% e %] {% e %}	list/vector of everything e leaves on the stack
p(% a %)	closure: p partially applied to a

A.2 Blocks — every opener has a closer

define f(a, b); ... enddefine;	if ... then ... elseif ... else ... endif
define f(a) -> r; ... enddefine;	unless ... then ... endunless
procedure(a); ... endprocedure	for i from 1 to n do ... endfor
define :class C; slot s = v; enddefine;	for x in list do ... endfor
define :method m(x:C); ... enddefine;	while c do ... endwhile
section \$-name => exports; ... endsection;	repeat ... quitif(c); ... endrepeat
exload tag [libs] ... endexload;	switchon e case < 0 then ... endswitchon

A.3 Declarations

vars permanent (global) — required for matcher pattern variables. lvars lexical local — the default choice inside a procedure. constant, lconstant their immutable forms. dlocal x = v rebind for the dynamic extent of the call, restored on *any* exit.

A.4 Mishap decoder

DECLARING VARIABLE x	(a warning) you used an undefined name — usually a typo or a missing <code>define</code>
NUMBER(S) NEEDED	arithmetic on a non-number, often the value of that undefined name
STRING NEEDED	a word "x" was passed where a string 'x' was wanted — check the quotes
ILLEGAL DECLARATION OF WORD	the name is already a protected system identifier; re-name
MISPLACED EXPRESSION ITEM	syntax: usually a missing keyword or a construct used with the wrong shape
EXECUTING NON-PROCEDURE	calling something that is not a procedure — often an undefined name
CAN'T RESTORE - NOT SAME SYSTEM AND VERSION	the .psv image predates the current <code>basepop11</code> ; rebuild the images
sysEXECUTE: NOT AT EXECUTE LEVEL	VM planting attempted at compile-time top level — wrap it in a procedure

A.5 Files, processes, tables

```
;;; read a file line by line
vars dev = sysopen('/path/file', 0, "line");
vars rep = line_repeater(dev, inits(4096));    ;;; buffer size = max line length
repeat
  rep() -> line;
  quitif(line == termin);
  if issubstring('ERROR', 1, line) then ... endif;
endrepeat;

sysobey('ls /tmp > /tmp/out');                ;;; run a command
vars r = sys_obey_linerep('set +e; grep pat file');    ;;; ...and stream it

newproperty([], 50, false, true) -> tbl;    ;;; hash table, keys matched by ==
newmapping([], 50, false, true) -> m;      ;;; ...keys matched by =
```

`line_repeater` truncates lines longer than its buffer, so size `inits(n)` generously. `sys_obey_linerep` runs its command line under `errexit`: prefix `set +e`; whenever a failure is expected.

A.6 Regular expressions

The escape character is `@`; `.` `*` `[ ]` `^` `$` are literal unless escaped; there is no alternation and no lookahead.

```
vars err, p;
regexp_compile('ERROR @[0-9@]{1,@}') -> (err, p);    ;;; test err -- false = ok
p(1, 'ERROR 42 happened', false, false) -> (i, n);    ;;; -> 1 8
substring(i, n, 'ERROR 42 happened') =>                ;;; ** ERROR 42
```

Case-insensitive: `regexp_compile(s, 1, false)`, or `@i` in the pattern. `p` is a compiled procedure — define it once, reuse it.

A.7 Running things

<code>./poplog pop11</code>	Pop-11 REPL
<code>./poplog basepop11 -target/psv/prolog.psv</code>	Prolog
<code>./poplog basepop11 -target/psv/clisp.psv</code>	Common Lisp
<code>./poplog basepop11 +target/psv/pml.psv</code>	Standard ML
<code>tools/forth.sh</code>	Forth
<code>./poplog basepop11 prog.p</code>	run a Pop-11 file
<code>tools/pop11-lsp tools/pop11-swank tools/pop11-mcp</code>	editor and agent servers
